

Pipelined RISC-V Processor Design

Naga Rama Hari Kumar
2024102045
ECE
IIIT Hyderabad
Spring 2026

Chandini Gayathri
2024102020
ECE
IIIT Hyderabad
Spring 2026

Veekshita Sai
2024102026
ECE
IIIT Hyderabad
Spring 2026

Abstract—This project implements a 5-stage pipelined RISC-V processor in Verilog. The design supports basic instructions and handles data and control hazards using forwarding, hazard detection, and pipeline flushing. The processor was verified through simulation.

Index Terms—RISC-V Processor, Pipeline Architecture, Data Hazards, Forwarding Unit, Hazard Detection, Verilog

I. INTRODUCTION

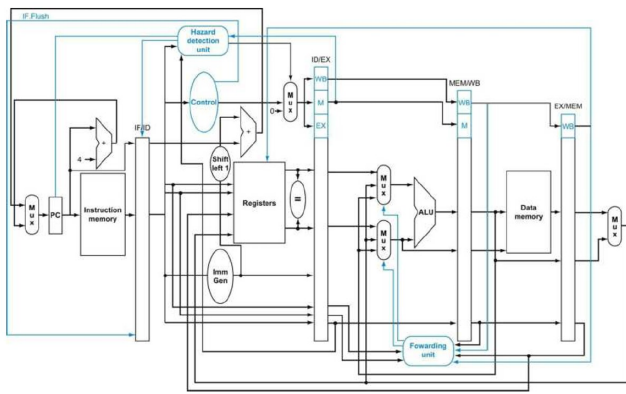


Fig. 1. Final Datapath and Control

Modern processors execute many instructions to run programs efficiently. In a basic sequential processor, each instruction completes all stages before the next begins, leading to inefficient hardware utilization. To improve performance, processors use pipelining, where instruction execution is divided into stages so multiple instructions can be processed simultaneously.

In this project, a five-stage pipelined RISC-V processor is designed and implemented using Verilog. The pipeline consists of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. To ensure correct execution, hazard handling mechanisms are implemented, including forwarding for data hazards, hazard detection for load-use dependencies, and pipeline flushing for branch instructions. The design is verified through simulation and waveform analysis.

II. SYSTEM OVERVIEW

This project implements a RISC-V processor using a modular, pipelined hardware design. The five-stage pipeline allows multiple instructions to execute simultaneously across different stages, significantly improving throughput over a sequential design.

A. Core Hardware Components

Program Counter (PC): Holds the address of the current instruction and updates every clock cycle to fetch the next one.

Instruction Memory: Stores the program and returns the instruction at the address the PC provides.

Register File: Provides 32 general-purpose registers supporting two reads and one write per cycle.

ALU: Performs arithmetic, logical operations, memory address computation, and branch condition evaluation.

Data Memory: Exclusively accessed by load and store instructions to read or write data.

Control Unit: Decodes the opcode and generates control signals that coordinate every component in the datapath.

B. Five-Stage Pipeline

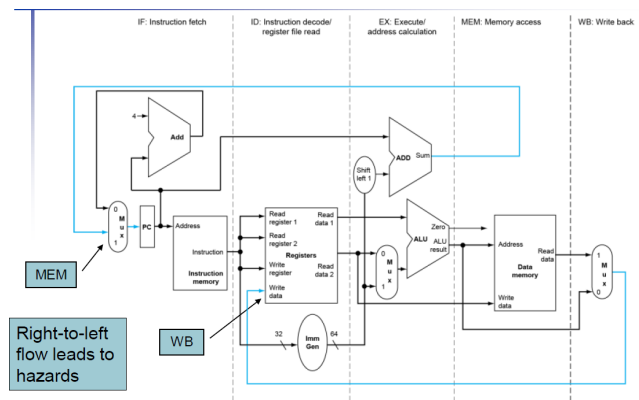


Fig. 2. RISC-V Pipelined Datapath

Instruction Fetch (IF): Fetches the instruction from memory using the PC and increments the PC by four for the next cycle. The instruction and PC+4 are stored in the IF/ID register.

Instruction Decode (ID): Decodes the instruction, reads source register values, sign-extends the immediate value, and generates all control signals. Results are stored in the ID/EX register.

Execute (EX): The ALU performs the required computation. For memory instructions, it calculates the effective address. For branch instructions, it evaluates the branch condition. Results move into the EX/MEM register.

Memory Access (MEM): Reads data memory for loads and writes to it for stores. Non-memory instructions pass their results through unchanged into the MEM/WB register.

Write-Back (WB): Writes the final result either the ALU output or data read from memory into the destination register, completing instruction execution.

MemWrite: Activates data memory write for store instructions.

MemToReg: Selects between memory-read data and ALU result as the value written back to the register file.

ALUSrc: Chooses between the Rs2 register value and the sign-extended immediate as the ALU's second operand.

ALUOp: Directs the ALU control logic to select the correct arithmetic or logical operation, derived together with the instruction's function fields.

Branch: Signals that the instruction is a branch. When the condition is satisfied, the PC is updated to the branch target address instead of PC+4.

C. Pipeline Registers

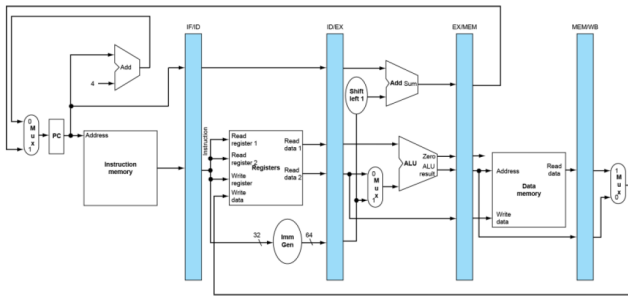


Fig. 3. Pipeline Registers

Four pipeline registers separate consecutive stages and carry intermediate data and control signals forward each clock cycle. Without them, outputs from one stage would immediately disturb the next, breaking the pipelined model entirely.

IF/ID: Holds the fetched instruction and PC+4 for the Decode stage.

ID/EX: Carries source register values, the sign-extended immediate, the destination register number, and control signals into the Execute stage.

EX/MEM: Passes the ALU result, branch comparison outcome, and memory control signals to the Memory Access stage.

MEM/WB: Forwards either the memory-read data or the ALU result, along with write-back control signals, to the Write-Back stage.

D. Control Unit and Control Signals

The control unit operates during the ID stage. Once the opcode and function fields are decoded, it generates the following signals which travel through the pipeline registers alongside the data:

RegWrite: Enables writing the result into the register file during Write-Back.

MemRead: Activates data memory read for load instructions.

E. Hazard Handling

Data Hazards: Occur when an instruction reads a register whose value a previous instruction has not yet written back a Read After Write (RAW) dependency. For example, `add x2, x1, x3` followed immediately by `sub x4, x2, x5` creates such a dependency on x2.

Forwarding Unit: Resolves most data hazards by routing results directly from later pipeline stages back to the ALU inputs, avoiding unnecessary stalls. Two forwarding paths are supported: EX/MEM to EX, and MEM/WB to EX.

Load-Use Hazard: Forwarding cannot help when a load is immediately followed by a dependent instruction, since the data only arrives after the MEM stage. For example, `ld x5, 0(x1)` followed by `add x6, x5, x2` creates this case. A hazard detection unit handles this by stalling the pipeline for one cycle freezing the PC, holding the IF/ID register, and inserting a NOP bubble so the loaded value is ready for forwarding in the next cycle.

Control Hazards: Arise because branch outcomes are only known at the end of the EX stage, by which point subsequent instructions have already been fetched. The processor uses static branch prediction, assuming branches are not taken and continuing to fetch sequentially.

Pipeline Flushing: If a branch turns out to be taken, the instructions fetched after it are invalid. These are removed by clearing the affected pipeline registers, inserting NOP bubbles in their place and redirecting the PC to the correct branch target address.

III. STAGE 1: INSTRUCTION FETCH UNIT

The Instruction Fetch (IF) unit forms the entry point of the five-stage RISC-V pipeline. Each clock cycle, it reads a 32-bit instruction from Instruction Memory at the address held by the Program Counter (PC), computes the address of the following instruction, and forwards both values to the next stage through the IF/ID pipeline register.

- 2) **Next-PC Computation:** A dedicated adder computes PC+4 in parallel with the memory read. If PCSrc = 1, a multiplexer selects the branch target address instead.
- 3) **Register Latch:** On the rising clock edge, both the fetched instruction and PC+4 are written into the IF/ID pipeline register, making them available to the Decode stage.

D. IF/ID Pipeline Register

The IF/ID register acts as a buffer between the Fetch and Decode stages, holding all data produced by the IF unit for exactly one clock cycle.

TABLE II
IF/ID REGISTER – STORED FIELDS

Field	Value Stored
Instruction	The 32-bit word fetched from Instruction Memory at address PC.
PC + 4	The incremented PC, used by the Decode stage for branch address calculation.

IV. STAGE 2: INSTRUCTION DECODE (ID)

Once an instruction is fetched, the processor needs to figure out what it actually means. That is the job of the Instruction Decode stage. It breaks the 32-bit instruction apart, reads the necessary register values, generates any immediate values, and produces the control signals that will guide the rest of the pipeline.

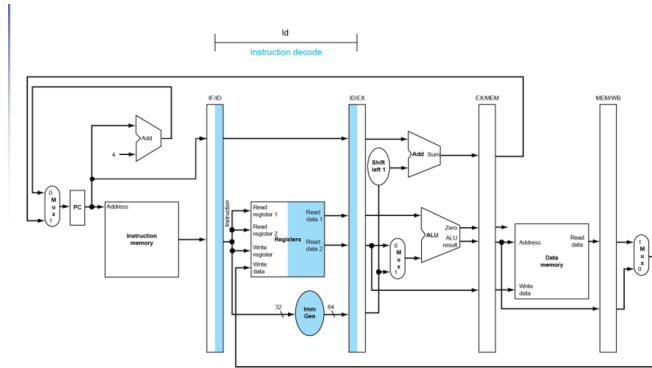


Fig. 5. ID for Load and Store

A. What This Stage Does

The ID stage is responsible for four things: extracting the opcode, function fields, and register addresses from the instruction; generating sign-extended immediate values for instructions that need them; reading operand values from the register file; and producing control signals for the Execute, Memory, and Write-Back stages. By the time this stage is done, everything the Execute stage needs is already prepared.

B. Key Modules

The ID stage is made up of several cooperating modules, each handling a specific part of the decode process. These are listed in Table III.

TABLE III
MODULES IN THE ID STAGE

Module	Role
Instruction Decoder	Pulls out the opcode, function codes, and register addresses from the instruction bits.
Register File	Holds the values of all general-purpose registers and supplies Rs1 and Rs2 for the current instruction.
Immediate Generator (imm_gen_all_types)	Extracts and sign-extends the immediate value according to the instruction format.
Control Unit (control_alucontrol)	Looks at the opcode and generates all necessary control signals for downstream stages.
ID/EX Pipeline Register	Latches all decoded data and control signals so the Execute stage can use them next cycle.

C. How the Decode Process Works

1) **Instruction Decoding:** The 32-bit instruction is split into named fields depending on its format. The opcode field identifies the instruction type whether it is R-type, I-type, Load, Store, or Branch. The Rs1, Rs2, and Rd fields point to the source and destination registers. The Funct3 and Funct7 fields further specify the exact operation for R-type and some I-type instructions.

2) **Immediate Value Generation:** Not all instructions use two register operands some encode a constant value directly inside the instruction word. The Immediate Generator handles this by extracting the relevant bits and sign-extending them to 32 bits. I-type instructions take bits [31:20] as the immediate. S-type instructions combine bits [31:25] and [11:7]. B-type branch instructions gather scattered bits and sign-extend them to form the branch offset.

3) **Operand Fetching:** The Register File reads the values stored at the Rs1 and Rs2 addresses identified during decoding. These values are the operands that the Execute stage will work with. Both reads happen in the same cycle, so there is no added delay.

4) **Control Signal Generation:** The Control Unit examines the opcode and produces a set of signals that travel alongside the data through the rest of the pipeline. These signals decide which ALU operation to perform, whether memory should be read or written, and whether the result should be written back to a register.

D. Control Signal Decisions

Table IV shows how the Control Unit sets each signal for the main instruction types.

TABLE IV
CONTROL SIGNAL DECISIONS BY INSTRUCTION TYPE

Opcode	Type	ALU Op	Mem Rd	Mem Wr	Reg Wr	ALU Src
0110011	R-type	10	0	0	1	0
0000011	Load	00	1	0	1	1
0100011	Store	00	0	1	0	1
0010011	I-type ALU	10	0	0	1	1
1100011	Branch	01	0	0	0	0

E. ID/EX Pipeline Register

At the end of the cycle, all decoded information is latched into the ID/EX pipeline register. This prevents the data from changing while the Execute stage is still working on it.

TABLE V
ID/EX PIPELINE REGISTER CONTENTS

Field	What It Holds
Read Data 1	Value read from the Rs1 register.
Read Data 2	Value read from the Rs2 register.
Immediate	Sign-extended immediate value, if the instruction uses one.
Control Signals	All execution, memory, and write-back control signals generated in this stage.

V. STAGE 3: EXECUTE (EX)

After decoding, the processor knows what operation to perform and which values to work with. The Execute stage is where the actual computation happens. It takes the operands passed through the ID/EX register and runs them through the Arithmetic Logic Unit (ALU), producing a result that either feeds into memory access or gets written directly back to a register.

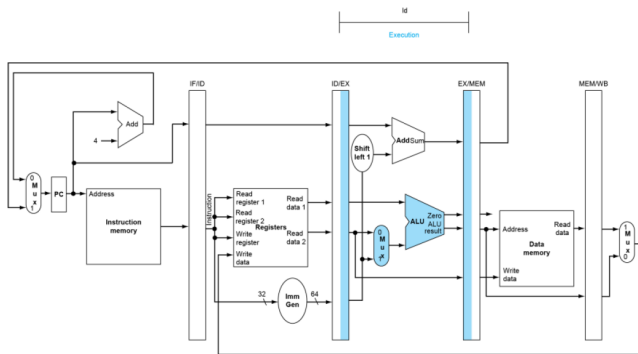


Fig. 6. EX for Load

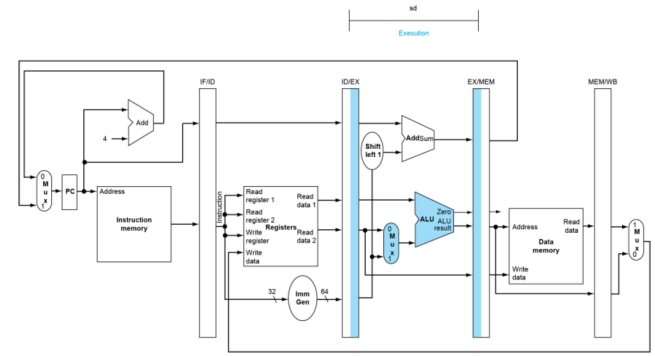


Fig. 7. EX for Store

A. Role of the ALU

The ALU sits at the heart of this stage. Depending on the 4-bit `op` control signal received from the Control Unit, it performs one of ten supported operations ranging from basic addition and subtraction to bitwise logic and shift operations. The result is a 64-bit value that moves forward to the MEM stage.

One important detail is how the second operand is chosen. Not every instruction uses two registers I-type and Load/Store instructions embed a constant directly in the instruction word. A multiplexer controlled by `idex_alusrc` handles this: when it is 1, the ALU uses the sign-extended immediate value; when it is 0, it uses the Rs2 register value read during decode.

B. Supported ALU Operations

Table VI lists every operation the ALU can perform, mapped to the 4-bit `op` code that selects it.

TABLE VI
ALU OPERATION ENCODING

op	Operation	Result
0000	ADD	$a + \text{operb}$
0001	SUB	$a - \text{operb}$
0010	AND	$a \& \text{operb}$
0011	OR	$a \text{operb}$
0100	XOR	$a \wedge \text{operb}$
0101	SRL	$a \gg \text{operb}[5:0]$ (logical)
0110	SLL	$a \ll \text{operb}[5:0]$ (logical)
0111	SRA	$a \ggg \text{operb}[5:0]$ (arithmetic)
1000	SLT	1 if $a < \text{operb}$ (signed), else 0
1001	SLTU	1 if $a < \text{operb}$ (unsigned), else 0

A small note on SLT and SLTU: since these are single-bit comparison results, they are zero-extended to 64 bits before being passed forward, so the result width stays consistent with all other operations.

C. Hardware Modules Inside the EX Stage

The EX stage is not a single block it is built from several specialised modules that each handle one operation type. Table VII summarises them.

TABLE VII
HARDWARE MODULES IN THE EX STAGE

Module	Function
ADD	64-bit addition with carry output.
SUB	64-bit subtraction.
AND_64_bit	Bitwise AND across all 64 bits.
OR_64_bit	Bitwise OR across all 64 bits.
XOR_64_bit	Bitwise XOR across all 64 bits.
SLR	Logical right shift by operb[5:0] positions.
SLL	Logical left shift by operb[5:0] positions.
SRA	Arithmetic right shift, preserving the sign bit.
SLT	Signed comparison; outputs 1 if $a < b$.
SLTU	Unsigned comparison; outputs 1 if $a < b$.

D. Signal Specification

Table VIII covers every signal that enters or leaves the EX stage.

TABLE VIII
EX STAGE SIGNAL SPECIFICATION

Signal	Dir.	Description
a	IN	First operand, taken from Rs1 via the ID/EX register.
b	IN	Second operand from Rs2, used when $index_alusrc = 0$.
imm	IN	Sign-extended immediate, used when $index_alusrc = 1$.
op	IN	4-bit signal from the Control Unit selecting the ALU operation.
index_alusrc	IN	MUX select: chooses between b and imm as the second operand.
ALU Result	OUT	64-bit computed value forwarded to the MEM stage.
Zero Flag	OUT	Set to 1 when the ALU result is zero; used to evaluate branch conditions.

E. Branch Condition Evaluation

One responsibility of the EX stage that often goes unmentioned is branch resolution. When a `beq` instruction reaches this stage, the ALU subtracts the two operands. If the result is zero, the Zero Flag is asserted. This flag travels to the control logic, which then decides whether to redirect the PC to the branch target or let it continue to $PC+4$. Resolving branches here rather than later reduces the number of pipeline stages that need to be flushed on a taken branch.

F. EX/MEM Pipeline Register

Once the ALU finishes, its outputs are captured in the EX/MEM pipeline register along with the control signals still needed by later stages.

TABLE IX
EX/MEM PIPELINE REGISTER CONTENTS

Field	What It Holds
ALU Result	The computed 64-bit output from the ALU operation.
Zero Flag	Indicates whether the branch condition was met.
Control Signals	Memory access and write-back control bits passed forward unchanged.

VI. STAGE 4: MEMORY ACCESS (MEM)

Not every instruction touches memory, but the ones that do — loads and stores pass through this stage to get that work done. The Memory Access stage sits between the Execute and Write-Back stages, and its job is straightforward: either read a value from data memory, write a value into it, or simply pass the ALU result forward unchanged if the instruction does not need memory at all.

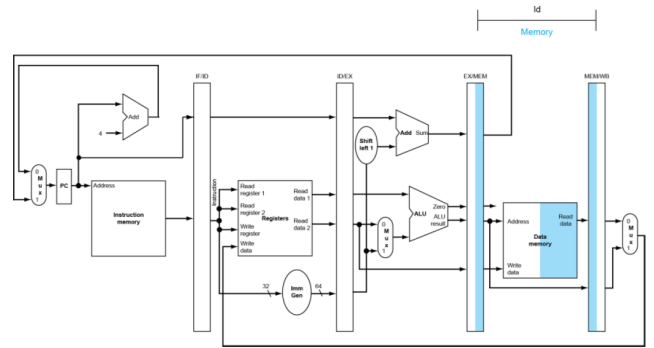


Fig. 8. MEM for Load

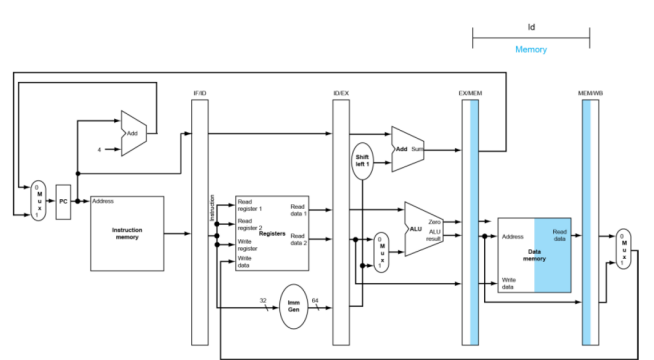


Fig. 9. MEM for Store

A. Why This Stage Exists

Data memory access cannot be folded into the Execute stage because reads and writes take their own clock cycle. Separating it into its own stage keeps the pipeline balanced and ensures that arithmetic instructions, loads, and stores all move through at the same rate without stalling each other unnecessarily.

B. How Memory Operations Are Decided

Two control signals arriving from the EX stage determine what this stage does. `idex_memread` being asserted tells the memory unit to fetch data from the address the ALU computed this is a load instruction. `idex_memwrite` being asserted tells it to write the Rs2 value into that same address this is a store. When neither signal is asserted, no memory operation takes place and the ALU result is simply passed through to the MEM/WB register.

It is worth noting that both signals are never asserted at the same time. The Control Unit ensures this by design, since reading and writing the same address in one cycle would produce undefined behaviour.

C. Key Modules

TABLE X
MODULES INSIDE THE MEM STAGE

Module	What It Does
Data Memory Unit	Performs the actual read or write to data memory at the ALU-computed address.
Memory Control Logic	Interprets <code>memread</code> and <code>memwrite</code> signals and drives the memory unit accordingly.
MEM/WB Pipeline Register	Captures either the memory read data or the ALU result, along with remaining control signals, for the Write-Back stage.

D. Load vs Store Behaviour

1) *Load Instruction (ld)*: When a load is executed, the ALU has already computed the target memory address by adding the base register value and the immediate offset. The Data Memory Unit reads from that address and places the retrieved value into the MEM/WB register. This value will eventually be written into the destination register during Write-Back.

2) *Store Instruction (sd)*: For a store, the ALU again provides the target address. The value held in Rs2 is written into data memory at that address. Since a store does not produce a result that goes into a register, nothing is forwarded to the Write-Back stage the pipeline simply moves on.

3) *Non-Memory Instructions*: R-type and I-type ALU instructions pass through this stage without touching memory. The ALU result from the previous stage is carried straight through to the MEM/WB register so Write-Back can store it in the destination register.

E. Control Signal Behaviour by Instruction Type

Table XI shows how the MEM stage behaves for each instruction type based on the control signals it receives.

TABLE XI
MEM STAGE BEHAVIOUR BY INSTRUCTION TYPE

Type	Mem Rd	Mem Wr	Action
Load (ld)	1	0	Reads from data memory at ALU-computed address.
Store (sd)	0	1	Writes Rs2 value to data memory at ALU-computed address.
R-type	0	0	No memory access; ALU result forwarded directly.
Branch (beq)	0	0	No memory access; branch already resolved in EX.

F. Memory Address Alignment

One aspect worth mentioning is memory alignment. In RISC-V, load and store addresses must be naturally aligned a 64-bit load must be at an address divisible by 8, a 32-bit load by 4, and so on. If the ALU produces a misaligned address, the processor raises an alignment exception rather than proceeding with the access. This is handled before the memory unit is activated.

At the close of this stage, the relevant data is latched into the MEM/WB register so the Write-Back stage has everything it needs.

TABLE XII
MEM/WB PIPELINE REGISTER CONTENTS

Field	What It Holds
Memory Read Data	The value fetched from data memory, valid only for load instructions.
ALU Result	The computed address or arithmetic result, used directly for non-load instructions.
Destination Reg (Rd)	The register number where the final result will be written during Write-Back.
Control Signals	Remaining write-back control bits that tell the WB stage what to do.

VII. STAGE 5: WRITE-BACK (WB)

The Write-Back stage is the final stop in the pipeline. After an instruction has been fetched, decoded, executed, and potentially sent through memory, this stage takes the result and writes it back into the register file so that future instructions can use it. Without this stage, all the computation that happened earlier would simply disappear nothing would ever be permanently stored.

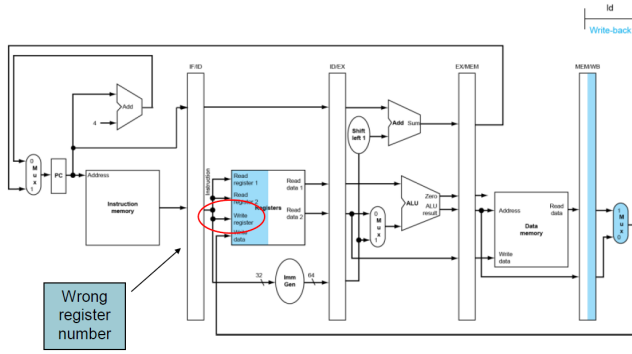


Fig. 10. WB for Load

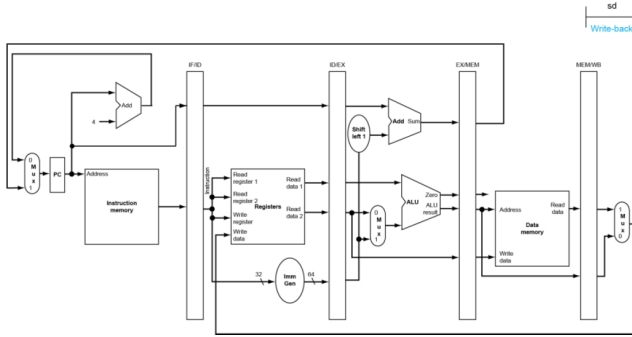


Fig. 11. WB for Store

A. MEM/WB Pipeline Register

B. What Gets Written Back

Not every instruction produces the same kind of result. An arithmetic instruction like `add` produces an ALU result that should go directly into the destination register. A load instruction, on the other hand, produces a value that came from data memory rather than the ALU. The WB stage needs to handle both cases correctly.

This is done using a multiplexer controlled by the `memtoReg` signal. When `memtoReg = 1`, the MUX forwards the memory read data to the register file this is the load case. When `memtoReg = 0`, it forwards the ALU result instead. Store and branch instructions do not write back at all since `RegWrite` remains deasserted for them.

C. Key Modules

TABLE XIII
MODULES INSIDE THE WB STAGE

Module	What It Does
Write-Back MUX	Selects between memory read data and ALU result based on the <code>memtoReg</code> control signal.
Register File	Receives the selected value and writes it into the destination register <code>Rd</code> .
Control Logic (RegWrite)	Acts as a gate — the write only happens when <code>RegWrite = 1</code> , preventing accidental register corruption.

D. Step-by-Step Operation

1) *Selecting the Result:* The MUX looks at `memtoReg` and picks the appropriate value. For loads, this is the data that the MEM stage fetched from memory. For all other register-writing instructions, this is the ALU result that has been carried forward through the EX/MEM and MEM/WB registers.

2) *Writing to the Register File:* The selected value is written into register `Rd` the destination register identified back in the Decode stage and carried through the pipeline in the MEM/WB register. This write happens at the first half of the clock cycle in most implementations, which is an important detail because the Decode stage reads the register file in the second half of the same cycle. This overlap means that a value written in WB can be read by a following instruction in ID within the same clock cycle, avoiding one potential data hazard without needing a stall.

3) *Instructions That Do Not Write Back:* Store instructions write to memory, not registers, so `RegWrite` is 0 and nothing is written. Branch instructions only update the PC, so they equally bypass this stage. The pipeline register simply holds their data harmlessly until the cycle ends.

E. Write-Back Decision by Instruction Type

TABLE XIV
WB STAGE BEHAVIOUR BY INSTRUCTION TYPE

Type	Reg Write	Mem to Reg	Value Written
R-type	1	0	ALU result written to <code>Rd</code> .
I-type ALU	1	0	ALU result written to <code>Rd</code> .
Load (ld)	1	1	Memory read data written to <code>Rd</code> .
Store (sd)	0	—	No register write occurs.
Branch	0	—	No register write occurs.

F. MEM/WB Register Used by This Stage

The WB stage does not have its own pipeline register at the output since it is the last stage. It consumes the fields that the MEM stage latched into the MEM/WB register.

TABLE XV
MEM/WB FIELDS CONSUMED BY WB STAGE

Field	How WB Uses It
Memory Read Data	Selected by MUX when memtoreg = 1; written to Rd for load instructions.
ALU Result	Selected by MUX when memtoreg = 0; written to Rd for arithmetic instructions.
Destination Rd	Tells the register file which register to write the final value into.
RegWrite	Enables the register file write port; if 0, no write takes place regardless of other signals.

VIII. HAZARD HANDLING IMPLEMENTATION

Pipelining keeps multiple instructions in flight at the same time, which is exactly what makes it fast. But that same overlap creates situations where one instruction needs a result that an earlier instruction has not yet finished producing. Left unhandled, these situations cause the processor to read stale register values and produce wrong answers silently. Three dedicated hardware units the Forwarding Unit, the Hazard Detection Unit, and the Branch Hazard Unit work together to catch and resolve every category of hazard before it causes an error.

A. Forwarding Unit

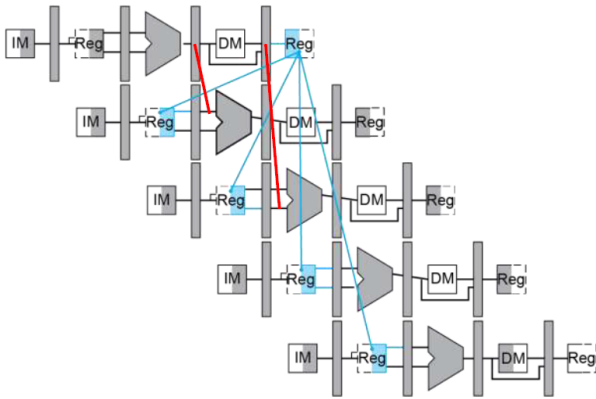


Fig. 12.

1) *The Problem It Solves:* When an instruction in the EX stage needs a value that a previous instruction is still carrying through the pipeline, waiting for that value to reach Write-Back would cost one or two unnecessary stall cycles. The Forwarding Unit avoids those stalls entirely by routing the result directly from wherever it currently sits EX/MEM or MEM/WB back to the EX stage input before the ALU needs it.

2) *How It Decides What to Forward:* The unit compares the source registers of the instruction currently in EX against the destination registers of the two instructions ahead of it in the pipeline. If a match is found and the ahead instruction is

writing back to a register, forwarding is applied. EX/MEM is checked first because it carries the most recent result.

TABLE XVI
FORWARDING UNIT SIGNAL SPECIFICATION

Signal	Dir.	Description
id_ex_Rs1	IN	Source register 1 of the instruction currently in EX.
id_ex_Rs2	IN	Source register 2 of the instruction currently in EX.
ex_mem_rd	IN	Destination register of the instruction in EX/MEM.
mem_wb_rd	IN	Destination register of the instruction in MEM/WB.
ex_mem_Regwrite	IN	Asserted when the EX/MEM instruction writes a register.
mem_wb_Regwrite	IN	Asserted when the MEM/WB instruction writes a register.
forwardA	OUT	MUX select for Rs1 operand in EX stage.
forwardB	OUT	MUX select for Rs2 operand in EX stage.

3) *Forwarding Priority and Encoding:* EX/MEM forwarding takes priority over MEM/WB because it carries the newer result.

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Fig. 13.

One edge case worth noting: if both EX/MEM and MEM/WB match the same source register, only the EX/MEM result is used since it is the more recent write. The MEM/WB match is ignored in this situation.

B. Hazard Detection Unit

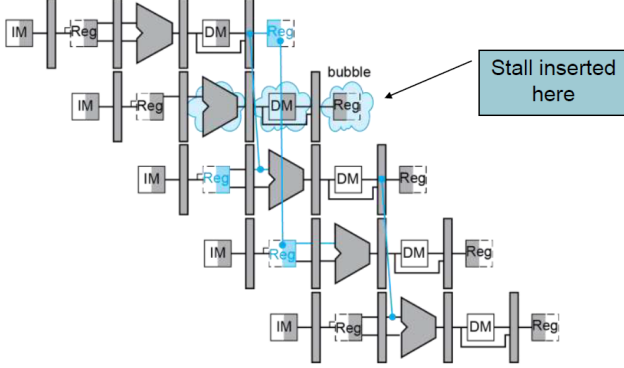


Fig. 14.

1) *The Problem It Solves:* Forwarding works well for ALU-to-ALU dependencies, but it cannot help when the producing instruction is a load. A load does not have its result until the end of the MEM stage, which is too late to forward back to an EX stage instruction that needs it in the same cycle. This load-use hazard requires a one-cycle stall there is no way around it.

2) *How It Works:* The Hazard Detection Unit watches the EX stage every cycle. If the instruction there is a load ($idex_memread = 1$) and its destination register matches either source register of the instruction currently in ID, a stall is inserted. The PC and IF/ID register are frozen for one cycle, and a NOP bubble is injected into the ID/EX register so the pipeline slot does not carry garbage forward.

TABLE XVII
HAZARD DETECTION UNIT SIGNAL SPECIFICATION

Signal	Dir.	Description
$idex_memread$	IN	High when the instruction in EX is a load.
$idex_rd$	IN	Destination register of the load instruction in EX.
$ifid_rs1$	IN	First source register of the instruction in ID.
$ifid_rs2$	IN	Second source register of the instruction in ID.
$stall$	OUT	Asserted to freeze the PC and IF/ID register for one cycle.

TABLE XVIII
HAZARD DETECTION DECISION TABLE

Condition	Stall	Reason
$idex_memread = 0$	0	No load in EX; no hazard exists.
$idex_memread = 1 \ \& \ idex_rd \neq ifid_rs1 \ \& \ idex_rd \neq ifid_rs2$	0	Load in EX but ID instruction does not use its result.
$idex_memread = 1 \ \& \ idex_rd = ifid_rs1$	1	Load-use hazard on Rs1; stall required.
$idex_memread = 1 \ \& \ idex_rd = ifid_rs2$	1	Load-use hazard on Rs2; stall required.

C. Branch Hazard Unit

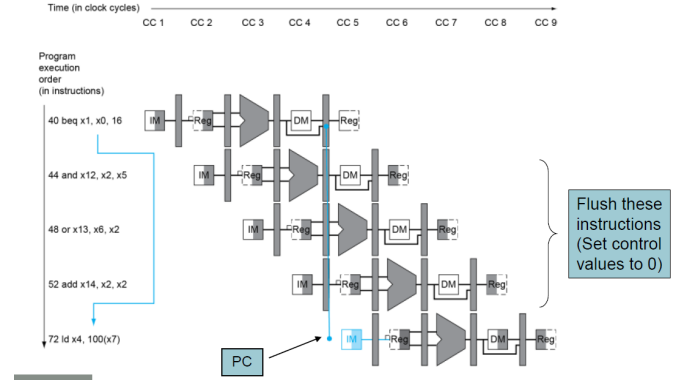


Fig. 15.

1) *The Problem It Solves:* Branch decisions are resolved in the EX stage, but by that point the IF and ID stages have already fetched and partially decoded the two instructions that follow the branch in sequential order. If the branch is taken, those two instructions are wrong and must be removed from the pipeline before they change any state.

2) *How It Works:* The Branch Hazard Unit monitors the branch and zero signals coming out of EX every cycle. When both are asserted together, meaning the branch instruction is a `beq` and the condition was met, the unit flushes the IF/ID and ID/EX pipeline registers by zeroing them out. This converts the two incorrectly fetched instructions into NOP bubbles. Simultaneously, pc_src is set to 1, redirecting the PC to the branch target address.

TABLE XIX
BRANCH HAZARD UNIT SIGNAL SPECIFICATION

Signal	Dir.	Description
branch	IN	Asserted when the instruction in EX is a branch.
zero	IN	ALU Zero Flag; high when the branch condition is satisfied.
clk	IN	Clock signal used to synchronise flush operations.
flush_ifid	OUT	Clears the IF/ID register, turning its instruction into a NOP.
flush_idex	OUT	Clears the ID/EX register, turning its instruction into a NOP.
pc_src	OUT	Selects next PC: 0 for PC+4, 1 for branch target address.

TABLE XX
BRANCH DECISION TABLE

Branch	Zero	Flush IF/ID	Flush ID/EX	PC Src	Action
0	0	0	0	0	Sequential execution continues.
0	1	0	0	0	Not a branch; Zero Flag irrelevant.
1	0	0	0	0	Branch not taken; continue normally.
1	1	1	1	1	Branch taken; flush pipeline and jump to target.

3) *The Cost of a Taken Branch:* Every taken branch wastes two cycles — the two instructions that were fetched speculatively and then flushed. This two-cycle penalty is a direct consequence of resolving branches in the EX stage rather than earlier. Some designs move branch resolution into the ID stage to reduce this to a one-cycle penalty, at the cost of added forwarding complexity into the decode logic.

IX. TESTBENCH AND SIMULATION SETUP

A Verilog testbench was written to verify the processor design. It initializes the processor, loads instruction sequences into instruction memory, and observes pipeline behavior over multiple clock cycles. Waveforms were captured and analyzed using GTKWave, which allowed signal transitions, pipeline activity, and control signals to be inspected in detail.

A. Instruction Input

Test programs were encoded using the RISC-V instruction subset supported by the processor and loaded directly into instruction memory at simulation start. The programs covered arithmetic operations, memory access, and branch instructions.

B. Verification Method

Key signals were monitored throughout simulation to confirm correct operation. These included the program counter, pipeline register contents, ALU outputs, register file updates, memory access signals, and all hazard-related control signals such as forwarding selects, stall enables, and branch flush signals.

X. EXPERIMENTAL RESULTS

The processor was tested across several instruction categories using simulation. Two types of outputs were analyzed:

- Terminal outputs showing the internal module results for the given test program.
- GTKWave timing diagrams showing pipeline signal activity.

These outputs confirm correct operation of the datapath modules, pipeline stages, forwarding logic, and hazard detection mechanisms.

A. Terminal Output Results

The following terminal outputs show the values produced by different processor modules during execution of the provided test program. These outputs were used to verify correct functionality of each module.

1) *Program Counter Output:* The terminal output below shows the program counter updating correctly during execution.

```
--- PC ---
VCD info: dumpfile pc.vcd opened for output.
After reset: pc_out=0 (expect 0)
pc_in=4:      pc_out=4 (expect 4)
pc_in=8:      pc_out=8 (expect 8)
stall=1:      pc_out=8 (expect 8, no update)
stall=0:      pc_out=12 (expect 12)
pc_tb.v:49: $finish called at 46 (1s)
```

Fig. 16. Terminal output showing program counter updates

2) *Instruction Decode and Immediate Generation:* The following output verifies that the immediate generator correctly extracts the immediate values.

```
--- IMMEDIATE_GEN ---
VCD info: dumpfile immediate_gen.vcd opened for output.
ADDI imm=10 (expect 10)
ADDI imm=-5 (expect -5)
SD imm=8 (expect 8)
BEQ imm=12 (expect 12)
immediate_gen_tb.v:36: $finish called at 40 (1s)
```

Fig. 17. Terminal output showing immediate value generation

3) *Control Unit Operation*: The control unit generates the appropriate control signals for each instruction.

```
--- CONTROL UNIT ---
VCD info: dumpfile control_unit.vcd opened for output.
R-type: RegWrite=1 MemRead=0 MemWrite=0 Branch=0 ALUSrc=0 MemtoReg=0 ALUOp=10
ADDI:   RegWrite=1 MemRead=0 MemWrite=0 Branch=0 ALUSrc=1 MemtoReg=0 ALUOp=11
LOAD:   RegWrite=1 MemRead=1 MemWrite=0 Branch=0 ALUSrc=1 MemtoReg=1 ALUOp=00
STORE:  RegWrite=0 MemRead=0 MemWrite=1 Branch=0 ALUSrc=1 MemtoReg=0 ALUOp=00
BRANCH: RegWrite=0 MemRead=0 MemWrite=0 Branch=1 ALUSrc=0 MemtoReg=0 ALUOp=01
control_unit_tb.v:45: $finish called at 50 (1s)
```

Fig. 18. Terminal output showing control signal generation

4) *Register File Operations*: The register file output confirms correct read and write operations.

```
--- REGISTER FILE ---
VCD info: dumpfile register_file.vcd opened for output.
Write x1=42, read x1=42 (expect 42)
x1=42 x2=100 (expect 42 100)
x0=0 (expect 0, always 0)
register_file_tb.v:51: $finish called at 36 (1s)
```

Fig. 19. Terminal output showing register read and write operations

5) *ALU Execution*: The ALU output demonstrates correct arithmetic and logical computations.

```
--- ALU ---
VCD info: dumpfile alu.vcd opened for output.
ADD 10 + 20 = 30 (expect 30)
SUB 20 - 10 = 10 (expect 10)
AND f0 & ff = f0 (expect f0)
OR f0 | f = ff (expect ff)
SUB 5 - 5 = 0, zero=1 (expect 0, zero=1)
alu_tb.v:44: $finish called at 50 (1s)

--- ALU CONTROL ---
```

Fig. 20. Terminal output showing ALU computation results

6) *ALU Control Unit*: The ALU control unit generates control signals to select the correct ALU operation.

```
--- ALU CONTROL ---
VCD info: dumpfile alu_control.vcd opened for output.
Load/Store: ALUOp=00 -> ctrl=0000 (expect 0000)
Branch:     ALUOp=01 -> ctrl=1000 (expect 1000)
R-ADD:      ALUOp=10 f3=000 f7=00000000 -> ctrl=0000 (expect 0000)
R-SUB:      ALUOp=10 f3=000 f7=01000000 -> ctrl=1000 (expect 1000)
R-AND:      ALUOp=10 f3=111 -> ctrl=0111 (expect 0111)
R-OR:       ALUOp=10 f3=110 -> ctrl=0110 (expect 0110)
ADDI:       ALUOp=11 f3=000 -> ctrl=0000 (expect 0000)
alu_control_tb.v:53: $finish called at 70 (1s)
```

Fig. 21. Terminal output showing ALU control signal generation

7) *Forwarding Unit Operation*: The forwarding unit output verifies that data hazards are resolved through forwarding.

```
--- FORWARDING UNIT ---
VCD info: dumpfile forwarding_unit.vcd opened for output.
TEST1 No hazard: ForwardA=00 ForwardB=00 (expect 00 00)
TEST2 EX hazard rs1: ForwardA=10 ForwardB=00 (expect 10 00)
TEST3 EX hazard rs2: ForwardA=00 ForwardB=10 (expect 00 10)
TEST4 MEM hazard rs1: ForwardA=01 ForwardB=00 (expect 01 00)
TEST5 MEM hazard rs2: ForwardA=00 ForwardB=01 (expect 00 01)
TEST6 EX priority: ForwardA=10 ForwardB=00 (expect 10 00)
TEST7 rd=x0 no fwd: ForwardA=00 ForwardB=00 (expect 00 00)
TEST8 RegWrite=0: ForwardA=00 ForwardB=00 (expect 00 00)
forwarding_unit_tb.v:107: $finish called at 80 (1s)
```

Fig. 22. Terminal output verifying forwarding logic

8) *Hazard Detection Unit*: The hazard detection unit correctly identifies load-use hazards and inserts stalls.

```
--- HAZARD DETECTION ---
VCD info: dumpfile hazard_detection.vcd opened for output.
TEST1 No hazard (MemRead=0): stall_pc=0 stall_if_id=0 flush_id_ex=0 (expect 0 0 0)
TEST2 Load-use rs1: stall_pc=1 stall_if_id=1 flush_id_ex=1 (expect 1 1 1)
TEST3 Load-use rs2: stall_pc=1 stall_if_id=1 flush_id_ex=1 (expect 1 1 1)
TEST4 No match (rd=x5): stall_pc=0 stall_if_id=0 flush_id_ex=0 (expect 0 0 0)
TEST5 rd=x0 no stall: stall_pc=0 stall_if_id=0 flush_id_ex=0 (expect 0 0 0)
hazard_detection_tb.v:74: $finish called at 50 (1s)
```

Fig. 23. Terminal output showing load-use hazard detection

B. GTKWave Timing Analysis

In addition to terminal outputs, GTKWave timing diagrams were used to analyze the dynamic behavior of the pipeline. These waveforms illustrate how instructions move through the five pipeline stages and how hazards are resolved during execution.

1) *ALU Instruction Execution*: The waveform below shows the execution of arithmetic instructions in the pipeline.

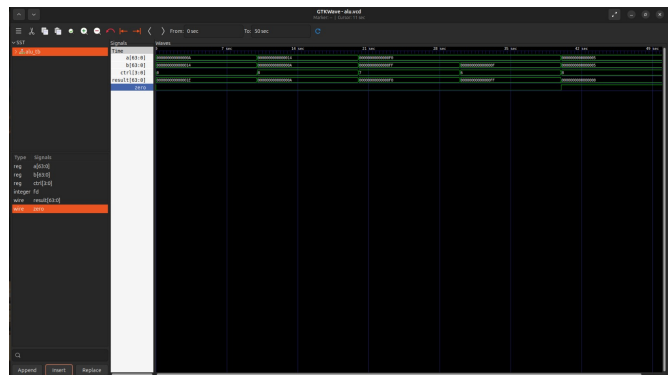


Fig. 24. GTKWave waveform showing execution of arithmetic instructions

2) *Control Unit Signals*: The control signals generated during instruction execution are shown below.

Advanced Execution Techniques: Superscalar execution, out-of-order scheduling, or deeper pipelining could all push throughput higher still, though each adds substantial design complexity.

XIII. CONCLUSION

A five-stage pipelined RISC-V processor was designed and implemented in Verilog, covering the Instruction Fetch, Decode, Execute, Memory Access, and Write-Back stages. Pipelining allows multiple instructions to execute simultaneously, improving throughput over a sequential design.

Hazard handling was a central part of the implementation. A forwarding unit resolves most data hazards without stalls. A hazard detection unit manages load-use cases by inserting a single stall cycle. Branch hazards are handled through static prediction and pipeline flushing when a branch is taken.

Simulation across arithmetic, memory, branch, and hazard test cases confirmed correct pipeline operation throughout. The project demonstrates that pipelining meaningfully improves processor performance while correct execution is preserved through careful hazard management.

XIV. TEAM CONTRIBUTIONS

Nagarama Hari Kumar

- Implemented the main processor integration module in `processor.v`.
- Designed the arithmetic logic unit in `alu.v`.
- Implemented the ALU control logic in `alu_control.v`.
- Assisted in preparing the project report and documentation.

Chandini Gayathri

- Implemented the pipeline register modules in `pipeline_regs.v`.
- Designed the forwarding unit for resolving data hazards in `forwarding_unit.v`.
- Implemented the control unit in `control_unit.v`.
- Contributed to writing sections of the project report and documenting the design.

Veekshita Sai

- Implemented the hazard detection unit in `hazard_detection.v`.
- Designed the register file in `register_file.v`.
- Implemented instruction memory and immediate generation modules in `instruction_mem.v` and `immediate_gen.v`.
- Implemented the program counter module in `pc.v`.
- Assisted in testing and verification of the pipelined processor.

REFERENCES

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*, Morgan Kaufmann Publishers, 2017.
- [2] RISC-V Instruction Set Manual, Volume I: Unprivileged ISA Specification.